

Flight Software Secure Programming

Steven Doran
NASA Jet Propulsion Laboratory

Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

Why Flight Software Secure Programming Matters

- Mission Critical
  - Spacecraft operate in an environment (Space) where software failures cannot be easily resolved
  - FSW is required to operate for longs period of duration w/o faults
  - A single vulnerability or FSW bug could render a multi-million / multi-billion dollar mission useless
- Remote Attack Surface
  - Spacecraft communicate via radio uplink and downlinks that can intercepted or spooked
  - Adversary can attempt to send malicious commands, inject false telemetry data, or disrupt communications
- Resource Constraints
  - FSW operates under limited hardware limitations
  - Poor programming practices could was hardware resources and create vulnerabilities attackers can use to exploit the FSW
  - Security must be implemented efficiently
- Radiation Environment
  - Space radiation generate Single Bit Errors (SBU) and Double Bit Errors (DBU)
  - Secure programming includes defensive techniques such as error detections, redundancy, and fault tolerance to ensure FSW can detect and recover from events

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Is This Code Safe?

```
void processCommand(const char* input) {
    char buffer[16];
    strcpy(buffer, input);
}
```

No!
Classic Buffer Overflow

```
void processCommand(const char* input) {
    char buffer[16];
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';
}
```

Truncate to prevent overflowing buffer w/ null terminator

```
void processCommand(std::string_view input) {
    // Safe truncation
    std::string username(input.substr(0, 15));
    // Or validate and reject:
    if (input.size() > 15)
        throw std::invalid_argument("Too long");
}
```

Better: Use std::string or std::array

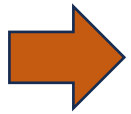
Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Let's Exploit the Code (Theoretical)

```
// Attacker's crafted input (simplified)
char malicious_input[] = "AAAAAAAAAAAAAAAA"  // 16 bytes - fills the buffer
"BBBBBBBB"                                   // 8 bytes - overwrites saved frame pointer
"\x40\x12\x40\x00"                           // 4 bytes - ove
                                             //              fire
"\x03\x00\x00\x00"                           // 4 bytes - thr
"\xFF\xFF\x00\x00";                          // 4 bytes - dur
```



```
void processCommand(const char* input) {
    char buffer[16];
    strcpy(buffer, input);
}
```



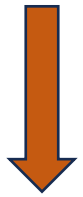
The attacker sends the malicious_input via the spacecraft's command uplink

- strcpy() copies malicious_input into buffer, overflowing the 16-byte buffer
- The return address contained in malicious_input is written into the stack (buffer is a local variable) to point to fireThruster()
- When processCommand() returns, instead of returning where expected, execution jumps to fireThruster()
- The overflow data inserts arguments on the stack to fireThruster()
- Thruster #3 fires for 65 seconds – Causing undesirable events on the Spacecraft

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Integer Overflow Vulnerabilities

```
void allocateBuffer(U32 count) {
    // Overflow Danger!
    U32 size = count * sizeof(int);
    int* buffer = (int*)malloc(size);
    .
}
```



```
#include <limits>

bool allocateBuffer(U32 count) {
    constexpr U32 max_count =
        std::numeric_limits<size_t>::max() / sizeof(int);

    if (count > max_count) {
        return False;
    }
```

```
// How an adversary could exploit this code
size is U32 (32 bits (4 bytes), max value is 4,294,967,295)
count = 1,073,741,824  // // Attacker provides huge count value (2^30)
// Size calculation
size = 1,073,741,824 * 4 = 4,294,967,296 // 4,294,967,296 > 4,294,967,295 (max 32-bit value), so it WRAPS AROUND to 0!)
size = 0  // Actual size after overflow
malloc(size) returns a small valid pointer that can be used to jump to adjacent heap objects
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Secure String Handling

```
// Avoid using (buffer overflows waiting to happen)
strcpy, strcat, sprintf, gets, scanf("%s", ...)
```

```
// Use instead
std::string, std::string_view
std::format (C++20) or fmt library

std::string name = std::format(
   ”Command: {}", command_input);
```

```
bool validateCommand(std::string_view command) {
   // Verify the command is not empty or larger than our allocated buffer size
   if (command.empty() || command.length() > 256) return false;

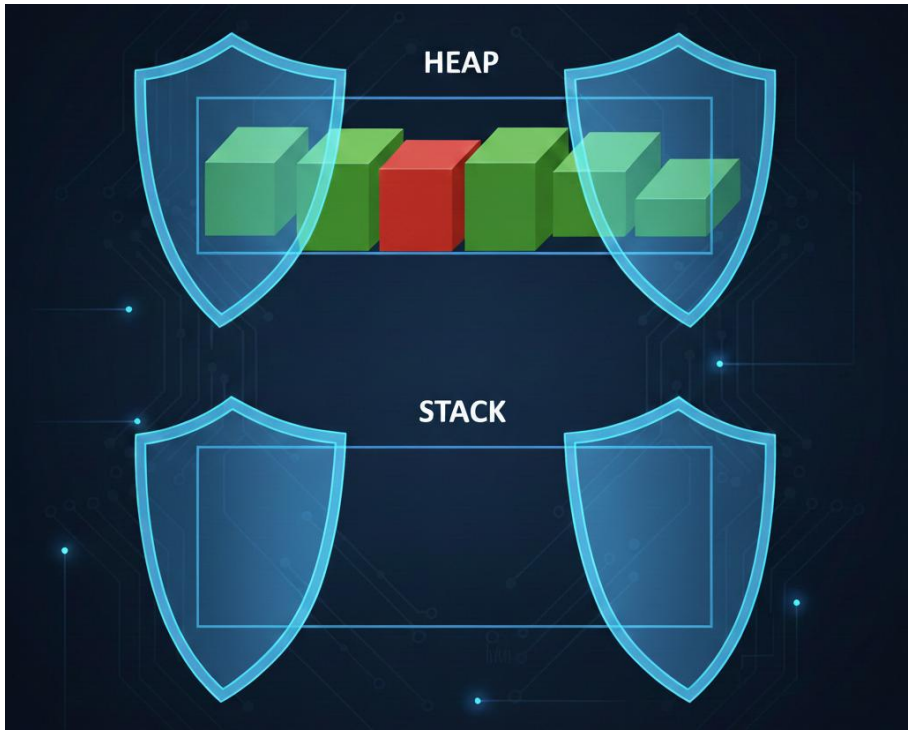
   // Process Command
   .
   .
```

Take Advantage of the Modern C++ Library (ex: Container Libraries std::string, std::array and std::vector)

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Protect the Stack and Heap

- Stack Overflows
  - Overwrite return addresses (hijack control flow)
  - Overwrite saved frame pointers
  - Overwrite other local variables defined in the function
  - Overwrite Function arguments
- Heap Overflows
  - Overwrite adjacent heap-allocated objects
  - Overwrite heap metadata (i.e. Malloc structures)
  - Overwrite Function pointers stored in heap objects



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Know and Understand C++ Undefined Behavior (There Be Dragons)

```
// Null Pointer Dereference
int* ptr = nullptr;
*ptr = 10;  // UB: Dereferencing null pointer
```

```
// Dangling Pointer (Use-After-Free)
int* ptr = new int(42);
delete ptr;
*ptr = 10;  // UB: Accessing freed memory
```

```
// Integer Overflow (Signed)
int maxInt = INT_MAX;  // 2,147,483,647
int result = maxInt + 1;  // UB: Signed integer overflow
```

```
// Shifting Beyond Bit Width
int x = 1 << 32;   // UB on 32-bit int
int y = 1 << -1;   // UB: Negative shift
```

```
// Use-After-Scope
int* getPointer() {
    int x = 42;
    return &x;  // UB: Returning pointer to local variable
}
int* ptr = getPointer();
*ptr = 10;  // UB: x no longer exists
```

```
// Sequence Point Violations
int arr[10];
int idx = 0;
arr[idx] = idx++;  // UB: Which idx is used for array access?
```

```
// Division by Zero
int x = 5 / 0;              // UB: Integer division by zero
double y = 5.0 / 0.0;  // NOT UB: Defined as infinity
```

Undefined Behavior occurs when a code enters a condition for defined in the C++ Standard

- FSW may appear to execute correctly, but has entered an unstable condition
- FSW may stop execution immediately or some period of time (Instability)

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Avoid Function Pointers

```
typedef void (*HandlerFunc)(U8 event);

static const HandlerFunc event_handlers[32] = {
   process_noop,             // 0
   process_mode_change, // 1
   process_fault             // 2
 };

void process_event(uint8_t event_id) {
   if (event_id <= 2) {
      // No type safety or bounds really enforced at
      // runtime beyond a simple check
      // Violates NASA Coding Rules
      event_handlers[event_id](event_id);
   }
}
```

```
// Use Switch Statement and Enums
typedef enum { EV_NOOP = 0, EV_MODE = 1, EV_FAULT=3,
EV_COUNT }
void process_event(uint8_t event_id) {
   switch (event_id) {
      case EV_NOOP : process_noop(event_id);   break;
      case EV_MODE : process_mode_change (event_id);  break;
      case EV_FAULT : process_fault (event_id); break;;
      default:
         // Safety-critical: always have explicit default!
         enter_safe_state();
         log_fault(EVENT_ID_OUT_OF_RANGE, event_id);
         break;
   }
}
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Static Analysis Tools

- Improves the Security Resistance of the FSW
  - Detects common security coding errors
  - Discovers software coding errors before the code is compiled
- Faster Hardware Integration Testing
  - Discovers many “hard to debug” coding issues before testing on the hardware
    - Hardware resource errors
    - Concurrency errors
- Standardizes Coding Standards for the development team
  - Impartial code reviewer
    - Treats all developers



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Real-World Consequences (Learn from Others)





Ariane 5 Explosion (1996)

Vulnerability: 64-bit floating point converted to 16-bit signed integer without overflow check

Impact:

Horizontal velocity exceeded 16-bit limit → integer overflow → corrupted inertial reference

Result:

Rocket veered off course 37 seconds after launch, self-destructed with $500M payload



Special Event

Vulnerability: Verification of proper protections from malicious commanding

Impact:

Adversary gained the ability to command of Spacecraft

Solution:

FSW Coding Change

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Resources

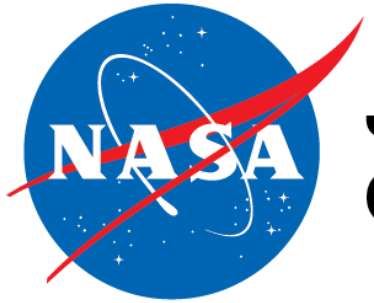
- CWE (Common Weakness Enumeration)
  https://cwe.mitre.org/

- CERT C++ Coding Standard
  https://wiki.sei.cmu.edu/confluence/x/fHQ

- C++ Core Guidelines
  https://isocpp.github.io/CppCoreGuidelines/

- OWASP Secure Coding Practices
  https://owasp.org/www-project-secure-coding-practices/

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Jet Propulsion Laboratory
California Institute of Technology